

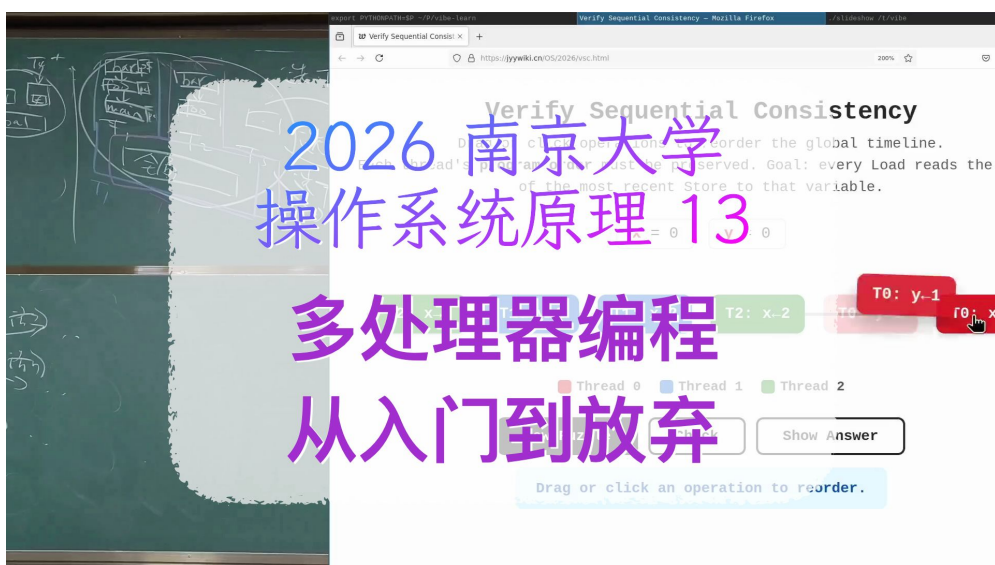
课程笔记

多处理器编程：从入门到放弃

2026 南京大学《操作系统原理》第 13 讲

lecture-to-notes

2026 年 4 月 18 日



视频作者/频道：绿导师原谅你了（蒋炎岩 / jyy）

发布日期：2026-04-14

视频时长：1:22:49

视频链接：<https://www.bilibili.com/video/BV1vgQGBREyJ/>

目录

1 本讲定位：从虚拟化走进并发	2
1.1 为什么需要并发	2
1.2 线程：共享内存的”进程”	3
1.2.1 并发 vs 并行	3
1.3 本章小结	4
2 入门：一个极简的线程库	5
2.1 两个 API: <code>spawn</code> + 自动 <code>join</code>	5
2.1.1 Hello 世界	5
2.1.2 线程安全的坑: <code>glibc</code> 里那些不能”thread-safe”的函数	6
2.2 动手实验：证明线程的基本性质	6
2.2.1 实验 1: 栈是独立的，但全局数据是共享的	6
2.2.2 实验 2: 线程栈到底有多大?	7
2.3 <code>gdb</code> 调试多线程: <code>set scheduler-locking on</code>	7
2.4 本章小结	8
3 放弃一：非确定性打破”1 + 1”	9
3.1 确定性 (determinism): 函数意义上的可预测	9
3.2 转账的经典 race condition	9
3.3 ”连 1 + 1 都不会算了”	9
3.3.1 最小值问题: NP-complete	10
3.4 并发破坏”计算机系统世界里的一切”	11
3.5 本章小结	11
4 放弃二：编译器和 CPU 背着你”乱来”	12
4.1 编译器: SimpleC 的”幻想 v.s. 现实”	12
4.2 致命陷阱: busy-wait on a shared flag	13
4.2.1 <code>-O0</code> 、 <code>-O1</code> 、 <code>-O2</code> 三种结果	13
4.3 CPU: 你真的有”shared memory”吗?	14
4.3.1 经典反直觉: (0, 0) 的结果	14
4.4 本章小结	15
5 总结与延伸	16
5.1 讲者核心观点	16
5.2 自测三问	16
5.3 与课程其他部分的 synthesis	16
5.4 拓展阅读	17

1 本讲定位：从虚拟化走进并发

上一讲（第 12 讲）把虚拟化部分收官：操作系统就是一组 API，加上层层叠叠的应用生态。本讲正式开启并发部分——从入门到”放弃”。

”放弃”到底是放弃什么

不是放弃并发编程本身——今天任何一个严肃的系统都逃不开并发。要放弃的是”顺序程序的确定性直觉”：

1. 放弃 状态迁移的确定性——一条语句执行下去会产生唯一结果；
2. 放弃语句即原子的模型——编译器会把你的代码重写；
3. 放弃共享内存即全局一致的模型——CPU 也会乱序；
4. 放弃”我多测几次都对，那就一定对”——并发 bug 只在你交付以后出现。

讲者本人的原话：”你们打开了一个魔鬼的盒子。”

1.1 为什么需要并发

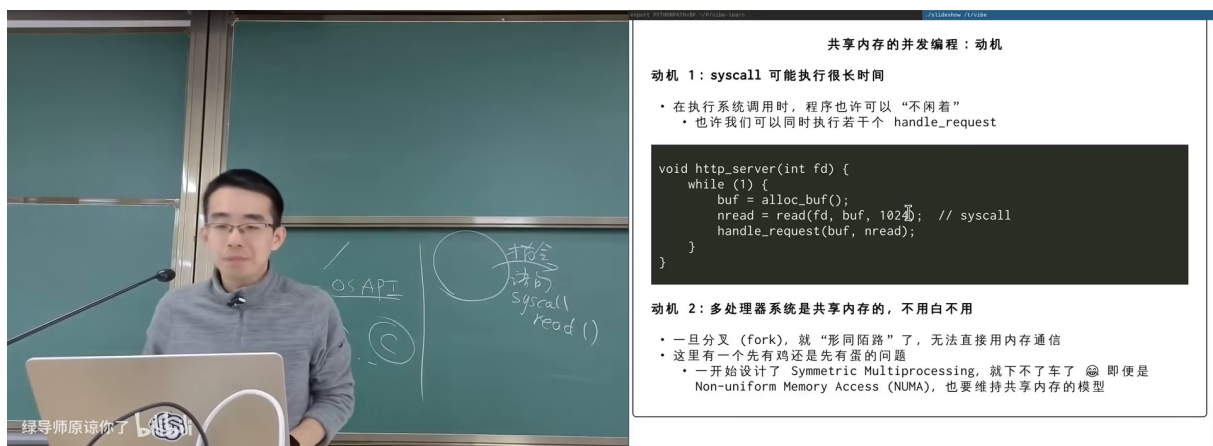


图 1：共享内存并发编程的两个动机：(1) syscall 会让程序卡住，哪怕手上还有别的事情可以做；(2) 现代硬件是多核共享内存，”不用白不用”——即使树莓派也有 4 核¹

两条主要动机：

- **异步 I/O 的需要**：写一个 HTTP server，如果是严格顺序执行，read 阻塞时整个程序就停住——哪怕此时还有别的请求可以处理。
- **多处理器的需要**：硬件上早就就是 SMP（对称多处理器），lscpu 看你的笔记本、服务器都是几十上百个核。物理上就是共享内存的，你不用就浪费。

¹视频画面时间区间：00:06:45–00:07:00。

fork 可以做”多核并行”吗？

可以——fork 100 次，每个子进程独立在不同 CPU 上跑，top 能看到 CPU 利用率 200%、300%。但收回结果很麻烦：要用命名管道或 IPC 把每个子进程算好的值送回父进程。如果所有 worker 能直接读写父进程的同一块内存（比如一个 `sum += x`），就能丢掉 IPC，也可以同时处理多个网络请求——这就是”共享内存并发”的需求。

1.2 线程：共享内存的”进程”

线程

把进程的地址空间画成「栈 + 全局」：

- 进程（单线程）：一个栈 + 全局；栈顶 frame 的 PC 驱动执行。
- 多线程进程：T1 / T2 / T3 各有一个独立的栈，但共享同一份全局变量区 + 代码段。

每次执行一条语句的含义变成”选一个线程、执行它栈顶 frame 的那条语句”。这就是 non-deterministic choice——魔鬼从这里开始。

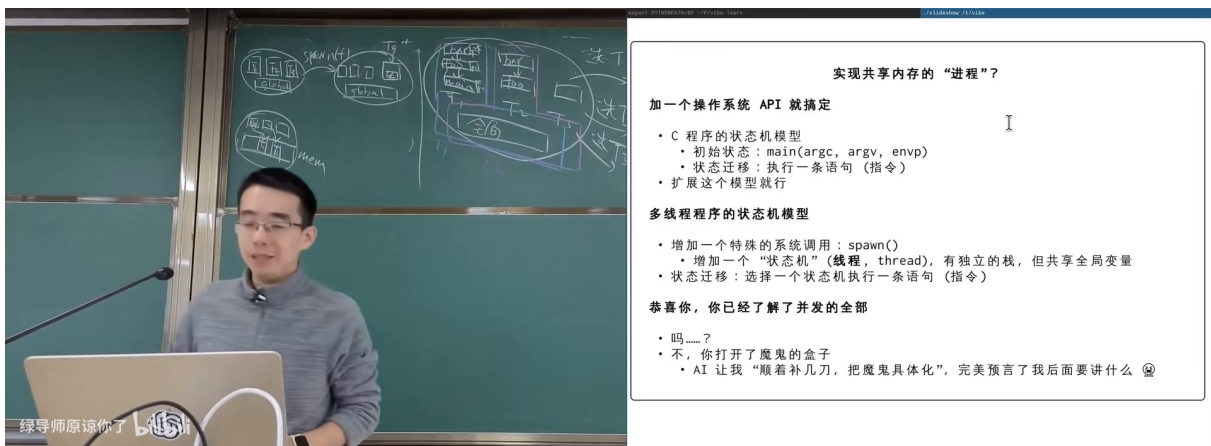


图 2：实现共享内存的”进程”——从 C 程序的执行模型（栈 + argc/argv/envp）扩展到多线程模型：每个线程有独立栈 + 共享全局；需要两个 API 即可：spawn(fn) 新建，exit 退出²

1.2.1 并发 vs 并行

- 并发 (concurrency)：逻辑上同时进行。单核也可以——CPU 在 A、B 之间来回切，从一个长时间尺度看两者都在”推进”。
- 并行 (parallelism)：物理上同时进行。必须有 2 个核同时跑两个线程。

蕴含关系

并行 $\Rightarrow$ 并发；反之不成立。并行是并发的一个子集——时间线上真的重叠了。

²视频画面时间区间：00:15:30–00:15:45。

1.3 本章小结

本章建立了**共享内存并发的概念模型**：线程 = 独立栈 + 共享全局。模型上看上去只是”多了个选线程”的动作，但这个”非确定的选”恰恰就是后面所有麻烦的根源。下一章把这个模型落到 C 代码上。

2 入门：一个极简的线程库

2.1 两个 API: spawn + 自动 join

讲者给了一个**最小可用**的线程库（对 POSIX pthread 的封装），只有两个接口：

```
1 // 新建一个线程，从函数 fn(tid) 开始执行；tid 从 1 开始编号
2 void spawn(void (*fn)(int tid));
3
4 // 在 main 返回时（或 join() 调用时），等所有 spawn 出去的线程结束
5 void join(void);
```

Listing 1: thread.h ——极简线程库

这是对 POSIX pthread_create / pthread_join 的”做减法”版本——POSIX 线程会要求手动管理线程句柄，这里统一隐藏。

为什么”基于标准”很重要？

POSIX pthread 是标准，**跨平台可依赖**——鸿蒙、FreeBSD 甚至某些 RTOS 都会实现它，哪怕内核实现和 Linux 很不一样。一旦某个 API 成为”事实标准”，你写的第三方库才能放心地跨平台分发。

2.1.1 Hello 世界

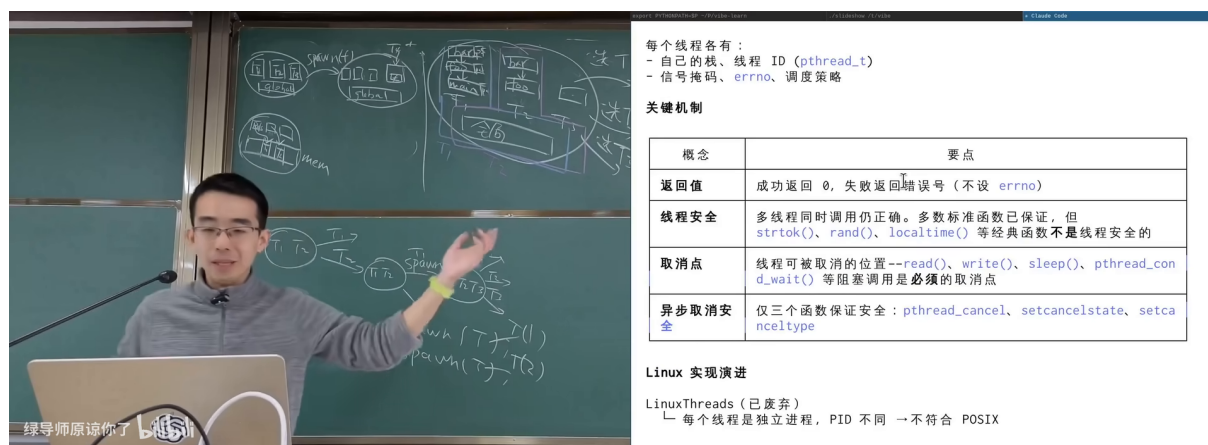
```
1 #include "thread.h"
2 #include <stdio.h>
3
4 void T_A(int tid) { while (1) printf("A"); }
5 void T_B(int tid) { while (1) printf("B"); }
6
7 int main() {
8     spawn(T_A);
9     spawn(T_B);
10    // join 会自动在 main 返回前执行
11 }
```

Listing 2: hello.c ——两个线程分别无限打印 A / B

运行一下就能看到 ABBAABABABBBAAABAB... 这样**随机交错**的输出。两个原因：

1. 操作系统调度时机不确定——什么时候给 T_A 让出 CPU？
2. 即使同时跑在两个核上，两核每条指令的执行速度不是一回事（缓存命中 / 分支预测 / 电压频率）。

2.1.2 线程安全的坑：glibc 里那些不能“thread-safe”的函数



每个线程各有：

- 自己的栈、线程 ID (pthread_t)
- 信号掩码、errno、调度策略

关键机制

概念	要点
返回值	成功返回 0，失败返回错误号（不设 errno）
线程安全	多线程同时调用仍正确。多数标准函数已保证，但 strtok()、rand()、localtime() 等经典函数不是线程安全的
取消点	线程可被取消的位置--read()、write()、sleep()、pthread_cond_wait() 等阻塞调用是必须的取消点
异步取消安全	仅三个函数保证安全：pthread_cancel、setcancelstate、setcanceltype

Linux 实现演进

LinuxThreads (已废弃)

- 每个线程是独立进程，PID 不同 一不符合 POSIX

图 3: pthread_attr_t 之外，更要紧的是 glibc 哪些函数是线程安全的：多个线程同时调用 strtok, rand, localtime 是不安全的（内部有静态缓冲区）；printf, fprintf, sleep, pthread_mutex_lock 是安全的，但行为可能微妙变化³

2.2 动手实验：证明线程的基本性质

2.2.1 实验 1：栈是独立的，但全局数据是共享的

```
1 int X = 0, Y = 0;
2 void T_X(int tid) { while(1) { X++; sleep(1); } }
3 void T_Y(int tid) { while(1) { Y++; sleep(2); } }
4
5 int main() {
6     spawn(T_X); spawn(T_Y);
7     while (1) { printf("X=%d Y=%d\n", X, Y); sleep(1); }
8 }
```

Listing 3: share.c ——一个线程 X++、另一个 Y++、第三个读

运行会看到 X 增速是 Y 的两倍——证明 X、Y 确实是所有线程共享的内存。类似地，你可以在一个线程里取一个局部变量的地址，传给另一个线程，另一个线程也能读到——说明栈虽然“属于某个线程”，但都在同一个地址空间里。

³视频画面时间区间：00:22:00–00:22:15。

2.2.2 实验 2：线程栈到底有多大？

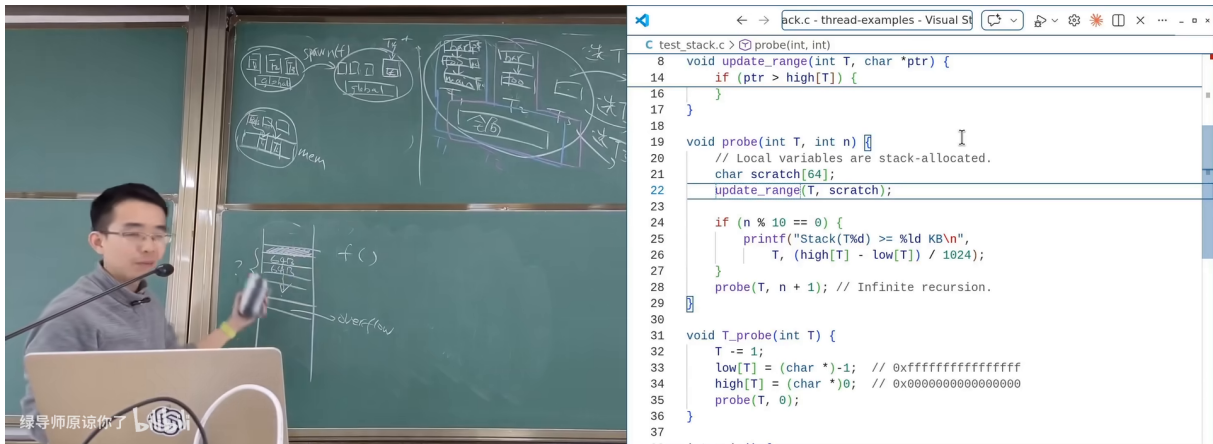


图 4: test_stack.c: probe 无限递归，每层分配 64 字节局部数组并更新记录“见过的最小和最大地址”；无限递归最终 stack overflow 前，能观测到线程栈的确切上下界。运行后发现每线程栈 $\approx 8192 \text{ KB}$ (8 MB)⁴

线程栈 主进程栈

主进程（单线程）的栈在 Linux 默认是“几乎无限增长”的——`ulimit -s` 默认是 8 MB 但带自动扩展。但**每一个线程的栈必须在地址空间里预先划定一段内存**，是一个固定大小的数组，默认 8 MB——用完就 stack overflow。这就是为什么高并发服务器里“一个连接一个线程”的模型吃内存特别狠：1 万个连接就占掉 80 GB 虚拟地址空间。

2.3 gdb 调试多线程：set scheduler-locking on

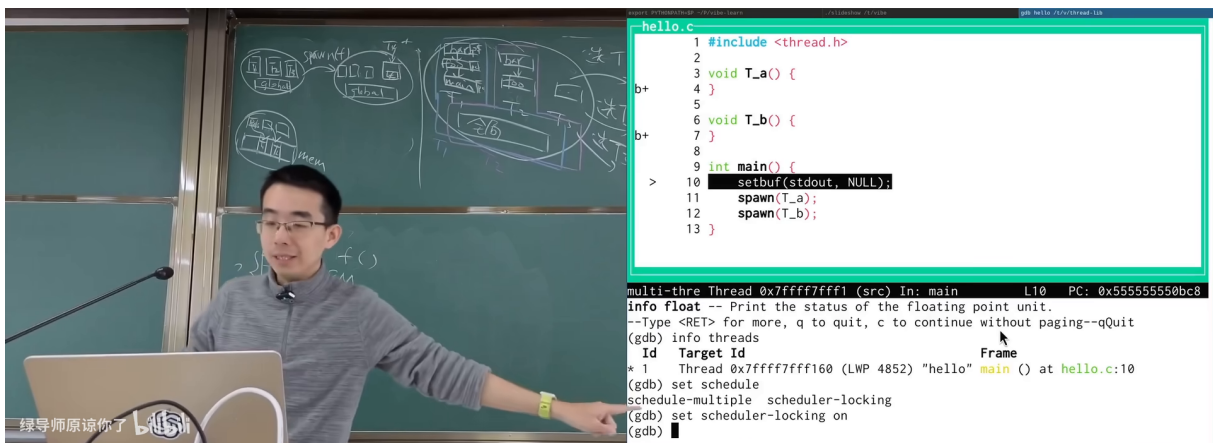


图 5: 用 gdb 调试 hello.c: info threads 看有几个线程；set scheduler-locking on 固定“step 时只动当前线程”，避免其他线程在你 step 的空隙里乱跑影响观察⁵

关键命令速查：

⁴ 视频画面时间区间：00:28:30–00:28:45。

⁵ 视频画面时间区间：00:33:00–00:33:15。

```
1 $ gcc -g -O0 hello.c -lpthread && gdb ./a.out
2 (gdb) break T_A           # 在线程入口打断点
3 (gdb) break T_B
4 (gdb) run
5 (gdb) info threads        # 列出当前所有线程
6 (gdb) thread 2            # 切到 2 号线程
7 (gdb) set scheduler-locking on # step 时只动当前线程
8 (gdb) step                # 精确走一步
9 (gdb) disassemble         # 看真实的汇编
```

Listing 4: 多线程 gdb 工作流

2.4 本章小结

你已经可以写出多线程程序、证明其共享内存性质、用 gdb 调试了。但正如讲者所说：“这就是多处理器编程的入门——你已经入门了。现在，是放弃的时候了。”

3 放弃一：非确定性打破“1 + 1”

3.1 确定性 (determinism)：函数意义上的可预测

什么是确定性？

数学的确定性：一个函数 $f: S \rightarrow S$ ，给定 s ， $f(s)$ 有且仅有一个值。程序的确定性：给定相同的初始状态，无论运行多少次，输出一定一样。

单线程程序（纯 C 不带 syscall）基本上是确定的——除了 `execve` 时 ASLR 注入的一些随机性、`read` 读到的键盘输入，程序本身的状态迁移就是纯数学函数 f 。

这就是 functional programming 被喜爱的根源

SICP / Haskell 里 *pure function* 的“**deterministically reproducible**”就是这种意义上的确定性。并发破坏了它。

3.2 转账的经典 race condition

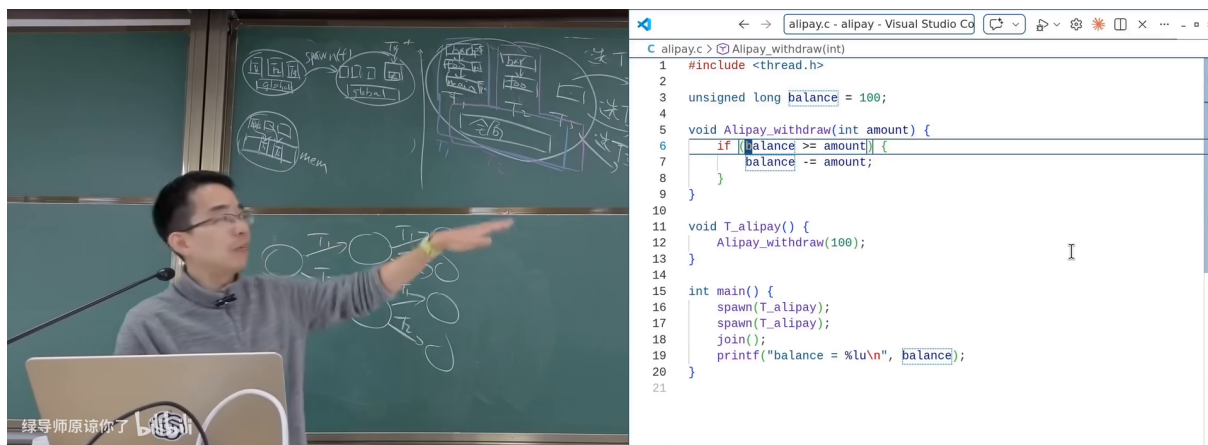


图 6: `alipay.c`: 两个线程同时 `Alipay_withdraw(100)`; `if (balance >= amount) balance -= amount`; 这段“check-then-act”在多线程下可以双双通过 `check`，然后两次都扣款，`balance` 变成 `-100`——再加上 `unsigned long`，结果是个天文数字⁶

真实世界“无限金币”的 bug 根源

讲者提到校园食堂某台打印机曾经可以这样把余额刷成负数——从此无限制消费。游戏里“复制物品”的 bug 也是同类的 race：点击地上物品 → 全局变量 `A` 被鼠标点击事件覆盖前，物品已经放进背包；然后同一个点击事件再次处理另一个同地址物品时，`A` 又被写一次，一份物品被指向两次。所有 check-then-act 路径都是 race 高危区。

3.3 “连 1 + 1 都不会算了”

两个线程各做 N 次 `sum++`，期望 $2N$ ？

⁶ 视频画面时间区间：00:42:30–00:42:45。

```

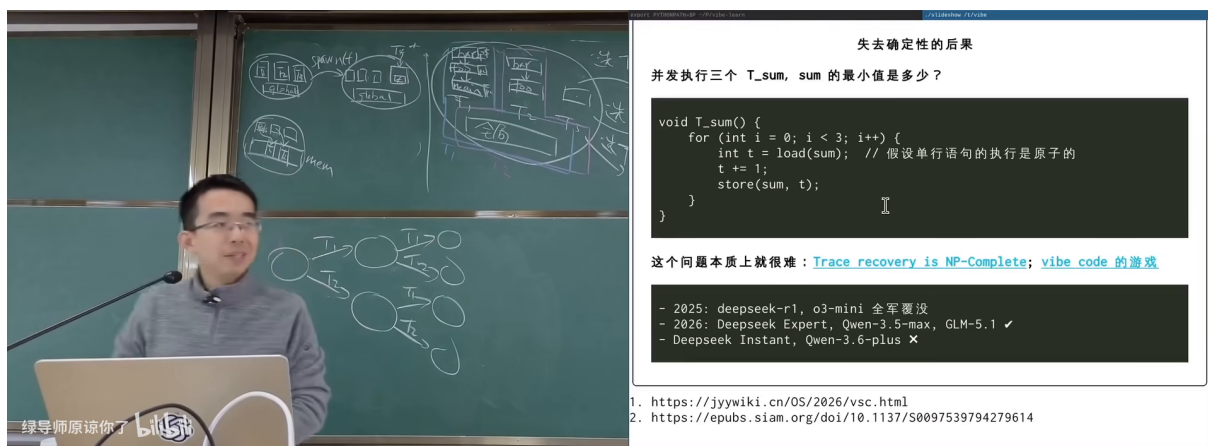
1 int sum = 0;
2 void T_sum(int tid) { for (int i = 0; i < N; i++) sum++; }
3
4 int main() { spawn(T_sum); spawn(T_sum); }

```

Listing 5: sum.c —— 并发递增

`sum++` 在汇编上是 `load sum` $\rightarrow$ `inc` $\rightarrow$ `store sum` 三步，两个线程”你读我还没写的老值”会互相覆盖，最终 `sum` 远小于 $2N$ ，而且每次都不一样。

3.3.1 最小值问题：NP-complete



失去确定性的后果

并发执行三个 `T_sum`, `sum` 的最小值是多少?

```

void T_sum() {
    for (int i = 0; i < 3; i++) {
        int t = load(sum); // 假设单行语句的执行是原子的
        t += 1;
        store(sum, t);
    }
}

```

这个问题本质上就很难: [Trace recovery is NP-Complete](#); [vibe code 的游戏](#)

- 2025: deepseek-r1, o3-mini 全军覆没
- 2026: Deepseek Expert, Qwen-3.5-max, GLM-5.1 ✓
- Deepseek Instant, Qwen-3.6-plus ✗

1. <https://jyywiki.cn/OS/2026/vsc.html>
 2. <https://pubs.siam.org/doi/10.1137/S0097539794279614>

图 7: Sum++ 最小值问题: 3 个线程每个做 N 次 `sum++` (`sum = load + 1; store`), 问 `sum` 的最小可能值? 讲者用它做过期中考试 OJ 题; 答案是 2。该问题本质上是”trace recovery”, 属于 NP-complete, Deepseek R1/O3-mini 当时做不对, 2026 年 Qwen3.5-Max/GLM-5.1 才能推理对⁷

最小 Sum

关键洞察: 让其中 $k - 1$ 个线程在读一个极早的值后睡觉——它们最后写入时把其他线程辛苦加出来的结果全覆盖掉。即:

1. 线程 *A*: `load sum=0` (不 `store`, 挂起)
2. 线程 *B*: `load sum=0` (挂起)
3. 线程 *C*: 完整做完 N 次 `sum++`
4. 线程 *A* 醒来 `store sum=1`, 覆盖 *C* 的结果
5. 线程 *B* 醒来 `store sum=1`, 再覆盖 *A*
6. 不对——更精细的构造下, 最终能把 3 个线程的全部工作”浪费”到只剩 2 次有效写入。

任意 k 个线程都可以用类似极端调度构造出 `sum = 2`。核心原理: ”check-then-act” 被 `load/store` 撕裂后, 几乎所有中间工作都可以被覆盖。

⁷ 视频画面时间区间: 00:48:25–00:48:40。

3.4 并发破坏”计算机系统世界里的一切”

printf 你熟悉吧？它内部有一个 buffer：

```
1 static char buf[4096];
2 static int pos = 0;
3
4 void printf(char *fmt, ...) {
5     // ... 展开 fmt 为字符串 str ...
6     for (char *p = str; *p; p++)
7         buf[pos++] = *p;           // 两个线程在这里碰撞！
8     if (should_flush) write(1, buf, pos);
9 }
```

Listing 6: glibc printf 的简化伪代码

两个线程都 printf 时 pos 就是一个 race——跟 sum++ 同样的病。幸好，glibc 的 printf 在实现里加了锁，所以你用它是安全的；但这也就意味着”所有系统调用和库函数都不得不为 thread-safe 付出代价”。

memcpy 却不是 atomic

即使标注”thread-safe”的 memcpy，也只是”不破坏内存”——两个线程同时 memcpy 到同一块目标，结果可能一半来自 A、一半来自 B。”线程安全”和”原子性”是两码事。

3.5 本章小结

并发把确定性的程序变成了非确定性图灵机——这和 P vs NP 里那种”non-deterministic Turing machine”在数学上是同一个东西。你会发现自己从”能写程序”退化到”连 1+1 都写不对”。

4 放弃二：编译器和 CPU 背着你”乱来”

即使你以为的两个线程是”我执行 `x++`；然后她执行 `y++`；”——**也不是**。你的代码根本不是按你写的那样执行。

4.1 编译器：SimpleC 的”幻想 v.s. 现实”

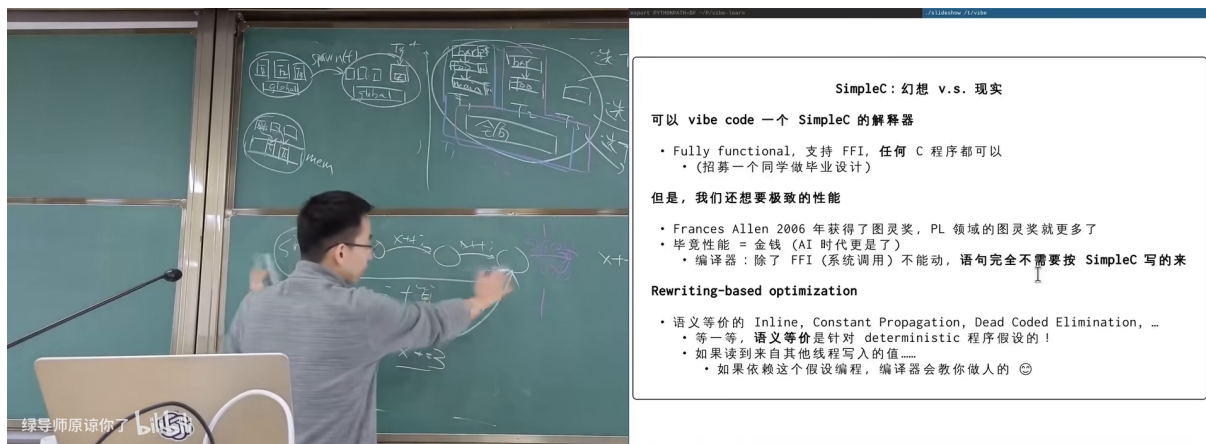


图 8: SimpleC 的幻想 vs 现实: 理论上可以 vibe-code 一个纯粹的 SimpleC 解释器 (按行执行); 实际上编译器 (GCC) 在 `.c` $\rightarrow$ `.s` 之间做了大量 **基于重写的优化**——inline、constant propagation、dead code elimination——所有这些假设的都是”程序顺序执行 + deterministic”⁸

编译器默认假设什么？

编译器假设：没有别人能修改我的内存。于是：

- 一连串 `x++`; `x++`; `x++`; 被合并成 `x += 3`;
- 循环里不变的表达式会被 **hoist** 到循环外;
- 永远不变的值会被 **cache** 到寄存器, 不再重读内存。

这些假设在单线程下 100% 正确——但多线程下**全部爆炸**。

⁸视频画面时间区间：01:01:30–01:01:45。

4.2 致命陷阱: busy-wait on a shared flag

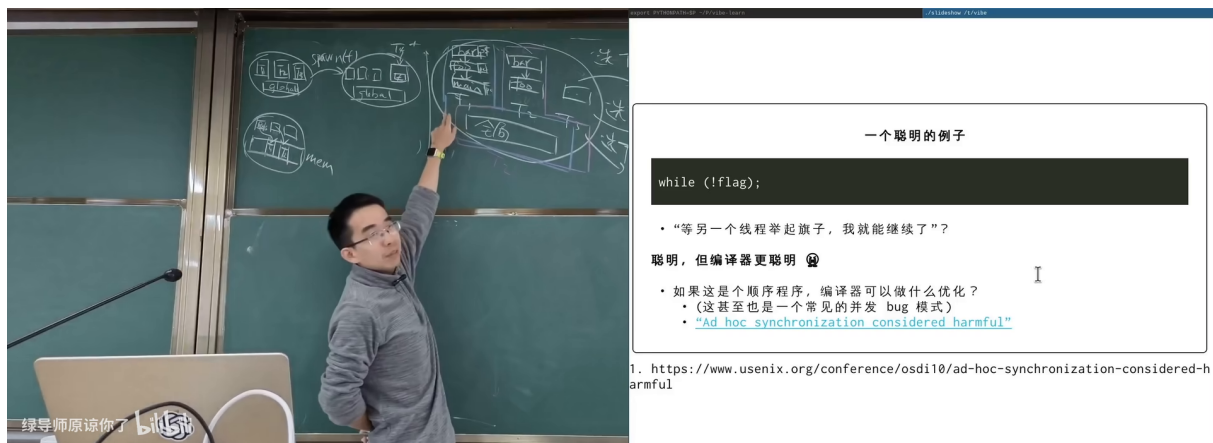


图 9: `while(!flag);` 等另一个线程把旗子举起来——在优化的编译器眼中，`flag` 在循环体内没被改过，所以它缓存进寄存器并把循环外提成 `if(!flag) while(1);`——死循环。这正是“Ad-hoc synchronization considered harmful”⁹

三种”看起来对、实际死”的写法

```

1 // 写法 1: 裸的 while 循环
2 while (!flag);           // 编译器优化成死循环
3
4 // 写法 2: 加一条 "yield", 可能还是死
5 while (!flag) usleep(100); // 好一点, 但 flag 仍可能被 hoist
6
7 // 写法 3: 正确
8 while (!__atomic_load_n(&flag, __ATOMIC_ACQUIRE));

```

4.2.1 -O0、-O1、-O2 三种结果

把 `sum++` 程序分别用不同优化级别编译：

```

1 $ gcc -O0 sum.c && ./a.out # 每次都得到随机的 < 2N 的数
2 $ gcc -O1 sum.c && ./a.out # 总是等于 N —— 为什么？
3 $ gcc -O2 sum.c && ./a.out # 奇迹般地等于 2N —— 更可怕！

```

Listing 7: 同一份代码，三种完全不同的行为

- O0**: 每次 `sum++` 都是 `load/inc/store` 三条指令，`race` 在每个 `++` 的中间频繁发生 → 结果混乱。
- O1**: 循环被优化成“循环前 `load` 一次 → 寄存器里 `N` 次 `+1` → 循环后 `store` 一次”。两个线程几乎同时 `load 0`，各自在寄存器里 `+N`，然后一前一后 `store N` → **最终 `sum = N`**。
- O2**: 循环整个被 `constant-fold` 成 `sum += N`——一条 `load`、一条 `store`。两条 `load` 和两条 `store` 之间相隔几纳秒；两个线程几乎不可能碰上。你测 100 万次都对，上线就 `panic`。

⁹视频画面时间区间：01:03:30–01:03:45。

最危险的 bug 是”看起来对的”

-O2 的情况是 race condition 的经典”幸存者偏差”表现——测试时永远观察不到，线上某一次诡异调度 + 高负载，bug 就出来了。”测了 100 万次都对”不是正确性证明。

4.3 CPU：你真的有”shared memory”吗？

没有——硬件层面也不是你想的那样。

现代处理器为了性能做了两件”背着你”的事：

1. **Out-of-order execution**: CPU 一个时钟周期可以 decode 好几条指令，基于数据流依赖动态重排。比如 `mov rax, 1; mov rbx, 2`；两条无依赖指令会被并行发射。
2. **Store buffer + cache coherency**: 每个核其实有自己的一份 L1 cache 和 store buffer。你 `store x=1` 时，值先在本核 store buffer 里，别的核暂时读不到，要等 cache 一致性协议传过去。

结果是”shared memory”根本不存在——真实模型是：每个 CPU 有一份自己的 **memory 副本**，硬件厂商只承诺**最终一致性**（eventual consistency）。

4.3.1 经典反直觉：(0, 0) 的结果

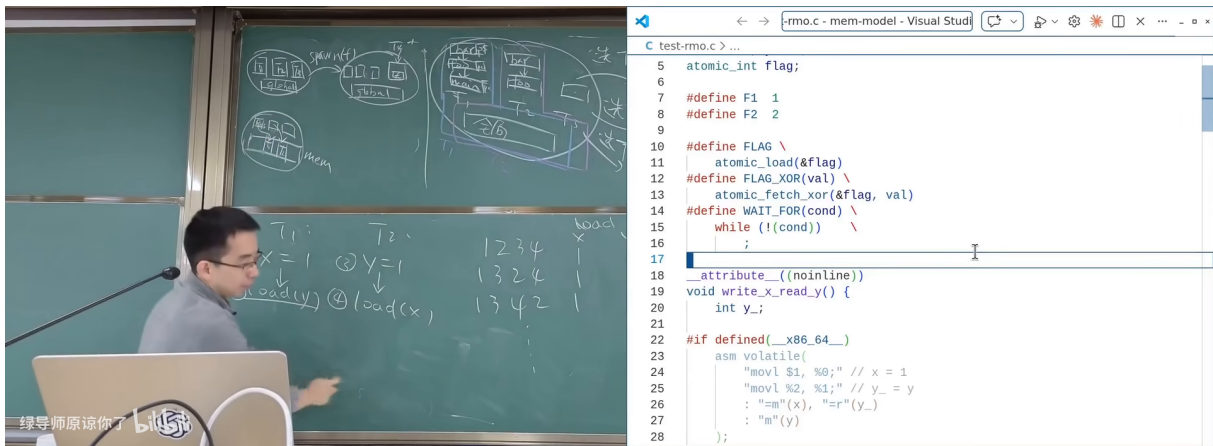


图 10: 内存模型测试：两个线程分别执行 `x=1; load y` 和 `y=1; load x`。在顺序一致性（SC）下， (x, y) 观察结果不可能是 $(0, 0)$ ；实际 x86 硬件上，跑 10 万次有约 6 万次是 $(0, 0)$ ——因为 store 还在各核的 store buffer 里就被”另一侧”的 load 越过了¹⁰

两线程 4 种交错 + 硬件重排

按 sequential consistency 模型，只存在 3 种合理结果： $(1, 0)$ 、 $(0, 1)$ 、 $(1, 1)$ 。现实中 $(0, 0)$ 占大多数——因为 x86 的 store buffer 让线程”先看到自己的 store，后让别人看到”。这告诉你：你写的 `x=1; load y`；在 CPU 那里可能已经变成 `load y; x=1`；的顺序。

¹⁰视频画面时间区间：01:18:30–01:18:45。

4.4 本章小结

”顺序程序的模型”在并发里全面崩塌：

- 编译器眼里，相邻语句可以合并、可以 hoist、可以整个抹掉；
- CPU 眼里，相邻无依赖指令可以乱序发射；
- Shared memory 在硬件层其实是 每核一份 + 异步同步。

所以讲者反复念叨一句话：***”Don’t play with shared memory.”*** 老老实实用后面要学的 mutex / atomic / lock-free 数据结构，让专家去填这些坑。

5 总结与延伸

5.1 讲者核心观点

1. **并发 = 共享内存的线程**——每个线程独立栈 + 共享全局，“选线程 + 执行一步”的 non-deterministic 模型。
2. **”从入门到放弃”放弃的是直觉**——不是放弃并发编程本身，而是放弃”顺序程序的确定性假设”。
3. **1 + 1 都算不对**：check-then-act race、sum++ race、printf buffer race...所有共享状态的并发修改都是坑。
4. **三层编译器在”帮倒忙”**：人脑编译器（sum++ 被想成一步）、gcc 编译器（-O1 变形）、CPU 编译器（乱序发射 + store buffer）——每一层都假设”别人不会改我的内存”。
5. **测试覆盖救不了你**：-O2 优化后的 race condition 在测试中可能永远观察不到，上线就出问题。
6. **现代系统的解药是 *Don't play with shared memory***：在 AI 时代，让 agent 自动化走”加锁 → 正确 → 优化”的路径；在关键路径上谨慎使用共享内存，其余地方用更安全的抽象。

5.2 自测三问

看完这讲，能答上这三题吗？

1. 用两个线程各做 N 次 `sum++`，写出能让最终 `sum = 2` 的调度序列（不限长度）；解释它为什么是 NP-complete。
2. 不用任何 `atomic` / `mutex`，写一段”`while(!flag);`” busy-wait 代码；用 `gcc -O2` 编译后看汇编，验证它被优化成了什么；给出至少两种改对的写法（提示：`volatile` / `__atomic_load_n`）。
3. 为两个线程的 `x=1; r=y; / y=1; r=x;` 写测试程序，在你的 x86 机器上跑 10 万次；统计 `(r1, r2)` 的分布，解释为什么会观察到 `(0,0)`。

5.3 与课程其他部分的 synthesis

- **与第 12 讲 (Linux from scratch)**：上一讲展示了 `OS = 对象 + API`，”没有魔法”。本讲揭示：单线程下”没有魔法”是对的，但并发引入的是**真正的 magic**——编译器 + CPU 会把你写的代码翻译成你看不懂的样子。
- **与第 5~7 讲 (进程、mmap、fd)**：本讲把”进程”复制了一份并改成”线程”，系统调用还是那批；但所有”进程内互不干扰”的保证现在都失效了。
- **与后续 14 讲 (并发控制：互斥)**：本讲提出问题，14 讲开始给答案——`pthread_mutex`、Peterson 算法、原子指令、`futex`——这是一整条”给魔鬼戴上笼头”的技术路径。

5.4 拓展阅读

- Erickson, *Ad-hoc Synchronization Considered Harmful*, OSDI 2010. <https://www.usenix.org/conference/osdi10/ad-hoc-synchronization-considered-harmful>
- McKenney, *Is Parallel Programming Hard, And, If So, What Can You Do About It?* (Perfbook, 开放文本) ——多处理器并发编程的百科全书式教材。
- Sewell *et al.*, *x86-TSO: A Rigorous and Usable Programmer's Model for x86 Multiprocessors*, CACM 2010 ——解释 x86 为什么会观察到 (0,0)。
- C11 / C++11 memory model: `_Atomic`, `memory_order_*`, `__atomic_*` builtins; `man 7 atomic`。
- Francis Allen, *The History of Language Processor Technology in IBM*, IBM Journal 2002; 2006 图灵奖得主关于编译优化的回忆。
- `man 3 pthread_create`、`man 7 pthreads`、`info '(gdb)Threads'`。